🧠 AutoGen Practical Quiz

🔹 Section A: Scenario-Based MCQs (Choose the BEST answer)

1. Multi-Agent Debugging System

You are building an AutoGen system where:

- Agent A: Writes Python code
- Agent B: Reviews and debugs
- Agent C: Executes code

The system is failing because Agent B is not correcting errors properly.

What is the BEST improvement?

A. Remove Agent B
B. Add prompt constraints to Agent B for structured feedback
C. Merge all agents into one
D. Increase token limit

Answer: B. Add prompt constraints to Agent B for structured feedback.
Reason: The reviewer agent needs a tighter role prompt with explicit review criteria like syntax check, bug detection, edge cases, and corrected code suggestions.

---

2. Conversational Agent Memory

A chatbot built using AutoGen forgets previous user inputs in a multi-turn conversation.

What should you implement?

A. Stateless API calls
B. Conversation memory or context window management
C. Reduce number of agents
D. Use only system prompts

Answer: B. Conversation memory or context window management.
Reason: Multi-turn systems need preserved chat history or memory so the agent can use earlier user inputs in later responses.

---

3. Agent Role Design

You created two agents:

- ResearchAgent
- SummaryAgent

Both are generating similar outputs.

What is the issue?

A. Token limit exceeded
B. Poor role separation in system prompts
C. API failure
D. Execution error

Answer: B. Poor role separation in system prompts.
Reason: If ResearchAgent and SummaryAgent have overlapping instructions, both will produce similar outputs instead of specialized work.

---

4. Infinite Loop Problem

Two agents keep responding to each other indefinitely.

What is the BEST fix?

A. Increase temperature
B. Add termination condition or max_turns
C. Add more agents
D. Reduce prompt size

Answer: B. Add termination condition or max_turns.
Reason: Multi-agent chats need a stop rule so agents do not keep replying forever.

---

5. Tool Usage in AutoGen

An agent needs to fetch real-time stock data.

What is the correct approach?

A. Hardcode stock values
B. Integrate tool/API calling capability
C. Use static prompts
D. Disable conversation

Answer: B. Integrate tool/API calling capability.
Reason: Real-time stock data must come from an external API or tool, not from static prompts or hardcoded values.

◆ Section B: Code-Based Scenarios

6. Identify the Issue

```
assistant = AssistantAgent(name="assistant")
user = UserProxyAgent(name="user")

user.initiate_chat(assistant, message="Write a function to sort a list")
```

The assistant gives code but it is never executed.

What is missing?

Answer: A code execution configuration or executor is missing.
You need a UserProxyAgent with code_execution_config enabled, or a dedicated execution agent/docker executor, so generated code can actually run.

---

7. Fix the Collaboration Flow

```
agent1 = AssistantAgent(name="coder")
agent2 = AssistantAgent(name="reviewer")

agent1.initiate_chat(agent2, message="Write code for factorial")
```

Problem: Reviewer is not giving meaningful feedback.

What change would you make in agent design?

Answer: Redesign the reviewer agent with a strict review-focused system prompt.
Example change: define it to check logic, edge cases, code quality, and return structured feedback with issues + fixes instead of writing fresh code.

---

8. Execution Agent Issue

```
user_proxy = UserProxyAgent(
    name="user",
    code_execution_config=False
)
```

Why will this fail in a coding workflow?

Answer: It fails because code execution is disabled.
With code_execution_config=False, the UserProxyAgent can chat, but it cannot run the generated Python code in the workflow.

---

9. Add a Third Agent

You have:

- Coder Agent
- Executor Agent

Now you want validation before execution.

What type of agent should you introduce and where in flow?

Answer: Add a Reviewer or Validator Agent between Coder and Executor.
Flow: User/Planner → Coder → Reviewer/Validator → Executor. This ensures correctness and safety checks before execution.

🔹 Section C: Design-Based Questions (Short Answers)

10. Multi-Agent Architecture Design

Design an AutoGen system for:
👉 “Automated Resume Screening”

Include:

- Agent roles
- Interaction flow

Answer:
Agent roles:
• Resume Parser Agent – extracts education, skills, experience, and keywords.
• Matching Agent – compares candidate profile with job description.
• Scoring Agent – assigns fit score based on weighted criteria.
• Screening Summary Agent – generates recruiter-friendly decision notes.
Interaction flow:
1. Input resume + job description.
2. Parser extracts structured candidate data.
3. Matching agent evaluates alignment.
4. Scoring agent ranks candidate.
5. Summary agent produces shortlisted/reject/review recommendation.

---

11. Conversational Agent Improvement

A chatbot gives generic answers and lacks personalization.

How would you improve it using AutoGen?

Answer:
Improve it by introducing specialized agents plus memory.
• Profile/Memory Agent stores user preferences, previous questions, and learning context.
• Response Agent generates tailored answers using that stored context.
• Critic Agent checks whether the reply is too generic and asks for personalization improvements.
Also maintain conversation history, user profile fields, and retrieval from prior interactions for better continuity.

---

12. Agent Collaboration Strategy

You are building a **Research Assistant** using:

- Web search
- Summarization
- Report generation

How will agents collaborate step-by-step?

Answer:
Step-by-step collaboration:
1. Query Router Agent receives the user request.
2. Web Search Agent collects relevant sources.
3. Summarizer Agent condenses source content into key points.
4. Report Writer Agent converts the summary into a structured report.
5. Review Agent verifies factual consistency, removes duplication, and improves clarity.
6. Final response is returned to the user.

---

13. Error Handling Scenario

During execution, one agent generates invalid Python code.

How should the system handle this collaboratively?

Answer:
The system should use collaborative recovery:
1. Executor Agent returns the exact error trace.
2. Debug/Reviewer Agent analyzes the failure.
3. Coder Agent revises the code based on the error.
4. Validator Agent checks the fix before re-execution.
5. Retry with limited attempts and terminate with a fallback explanation if errors persist.

🔹 Section D: Advanced Practical Scenarios

14. Real-World Workflow

Build a system for:
👉 “Automated Data Analysis from CSV”

What agents will you create and what will each do?

Answer:
Recommended agents:
• CSV Ingestion Agent – loads CSV, validates schema, handles missing values.
• Data Cleaning Agent – preprocesses nulls, duplicates, and type issues.
• Analysis Agent – computes statistics, trends, correlations, KPIs.
• Visualization Agent – generates charts/tables.
• Insight Agent – explains business insights from analysis.
• Report Agent – prepares final summary/report.

Flow: Upload CSV → ingestion → cleaning → analysis → visualization → insights → final report.

---

15. Prompt Engineering Challenge

An agent gives vague answers.

Rewrite the agent's system prompt to:

- Improve clarity
- Ensure structured output

Answer:
Improved system prompt:
"You are a precision-focused assistant. Answer clearly and concisely. Always respond in this structure:
1. Objective
2. Key steps
3. Final answer
4. Assumptions/limitations
If code is required, provide commented code only after explaining the logic. Do not give vague statements. Be specific, actionable, and technically correct."

---

16. Multi-Agent Decision System

You want agents to **vote on best solution**.

How will you design this?

Answer:
Design a multi-agent voting layer:
1. Multiple solver agents independently generate solutions.
2. Evaluator agents score each solution on correctness, efficiency, and safety.
3. A Judge/Coordinator Agent aggregates the votes.
4. Choose majority vote or weighted voting based on agent expertise.
5. If tied, trigger a critic round or human review.

---

17. Scaling Issue

Your AutoGen system becomes slow with multiple agents.

What optimizations would you apply?

Answer:
Optimizations:
• Reduce unnecessary agent turns.
• Use strict role separation to avoid duplicate work.
• Add termination conditions and max_turns.
• Cache repeated prompts/results.
• Use smaller/faster models for simple tasks and stronger models only where needed.
• Parallelize independent subtasks.
• Limit context size with summarization/memory compression.

---

18. Security Scenario

An agent executes user-provided code.

What safeguards will you implement?

Answer:
Safeguards:
• Sandbox execution environment.
• Restrict filesystem/network access.
• Allowlist safe libraries and commands.
• Set CPU, memory, and time limits.
• Scan code for dangerous operations before execution.
• Log all actions and outputs.
• Add human approval for risky tasks.

🔹 Section E: Mini Case Study

19. End-to-End System Design

Build an AutoGen system for:

👉 **"AI Tutor for B.Tech Students"**

Requirements:

- Answer doubts
- Generate quizzes
- Evaluate answers

Design:

- Agents involved
- Collaboration flow
- Memory handling

Answer:
Agents involved:
• Tutor Agent – explains concepts and answers doubts.
• Quiz Generator Agent – creates topic-wise quizzes.
• Evaluator Agent – checks student answers and gives feedback.
• Memory/Profile Agent – stores student level, weak areas, and past performance.
• Planner Agent – decides whether to teach, quiz, or evaluate.
Collaboration flow:
1. Student asks a doubt or requests quiz/help.
2. Planner routes the task.
3. Tutor answers conceptual questions.
4. Quiz Generator creates quizzes when requested.
5. Evaluator reviews submitted answers and returns score + explanation.
6. Memory agent updates student profile and weak topics.
Memory handling:
Store conversation history, topic mastery, quiz performance, and preferences so future answers become personalized and progressive.

---

20. Debug This Workflow

Flow:

1. User → Planner Agent
2. Planner → Coder
3. Coder → Executor
4. Executor → User

Problem: Output is incorrect and no validation happens.

Answer:
The workflow needs a validation layer.
Improved flow:
User → Planner Agent → Coder Agent → Reviewer/Validator Agent → Executor Agent → User
Why: The Reviewer/Validator checks logic, syntax, and expected output before execution. If issues are found, it sends feedback back to the Coder. Add retry limits and success criteria before final output is returned.